

Markov Decision Processes

CSC 480: Artificial Intelligence, Spring 2026

I Part I: Markov Decision Processes

Q1: ¹ (a) Define a Markov Decision Process. List all five components, give the mathematical notation for each, and explain what each one represents.

(b) Then write the distribution constraint that the transition model must satisfy for every state action pair (s, a) , and explain why it is necessary.

Q2: State the Markov property formally. What does it say about the relationship between the future state and the full history of past states and actions? Why does this property make MDPs tractable? What concept from Bayes Networks captures the same idea? ²

Contents

I	Part I: Markov Decision Processes	1
II	Part II: Horizons and Discounting	4
III	Part III: The Bellman Equation ..	6
IV	Part IV: Value and Policy Iteration	11
V	Part V: Reinforcement Learning	16

¹08-MDP-Bellman, Pages 1-2

²08-MDP-Bellman, Page 3

Q3: What is a *policy*? How does it differ from a *plan* as computed in a search problem? Explain the difference between the two as it pretains to MDP solution spaces.³

³08-MDP-Bellman, Page 4

Q4: Consider the following MDP describing a roomba operating in a home. The robot can be in one of four states: **Clean** (the floor is tidy), **Messy** (debris has accumulated), **Jammed** (wedged under furniture but potentially recoverable), or **Broken** (the robot has been irreparably damaged). The full transition and reward model is:⁴

State	Action	P' (Clean)	P' (Messy)	P' (Jam'd)	P' (Broke)	Reward
Clean	Patrol	0.7	0.3	0.0	0.0	+3
Clean	Charge	1.0	0.0	0.0	0.0	+1
Messy	Vacuum	0.6	0.35	0.05	0.0	+5
Messy	Explore	0.4	0.3	0.3	0.0	+8
Jammed	Reset	0.0	1.0	0.0	0.0	−5
Jammed	Force	0.5	0.0	0.0	0.5	+1
Broken	Wait	0.0	0.0	0.0	1.0	−20

⁴I made some abbreviations to make it all fit in the table

Explore sends the robot into unexplored territory: 40% of the time it finds a shortcut to a cleaner configuration, 30% of the time it finishes where it started, and 30% of the time it wedges itself under furniture. When *jammed* the robot has two recovery options. *Reset* where it carefully backs out, always landing in Messy (rewarded with −5 for the wasted effort) and *Force* where it tries to muscle free. 50% of the time it succeeds and lands in Clean, but 50% of the time it destroys a motor and the robot enters *Broken*. When *broken* the robot is beyond repair and is sad.

(a) Verify that the transition probabilities form a valid distribution for each state-action pair. Which state is *absorbing* (terminal)?

(b) For each non-absorbing state, identify the “safe” action and the “risky” action. Justify using the transition model. For Jammed specifically, note that Force has a positive immediate reward (+1) but non-zero Broken probability. Reset has a negative reward (−5) but a guaranteed non-catastrophic outcome. Why might Reset still be preferred at high γ ?

(c) When $\gamma \rightarrow 0$ ⁵ (completely myopic), what action does the agent take from each non-absorbing state? Show the Q-value comparisons that support each choice.

⁵Gen α ipad goblin

(d) For Messy, write $Q^*(\text{Messy}, \text{Explore})$ in terms of $V^*(\text{Clean})$, $V^*(\text{Messy})$, and $V^*(\text{Jammed})$. For Jammed, write both $Q^*(\text{Jammed}, \text{Reset})$ and $Q^*(\text{Jammed}, \text{Force})$ in terms of $V^*(\text{Clean})$, $V^*(\text{Messy})$, and $V^*(\text{Broken})$. Argue qualitatively which action dominates at Jammed as $\gamma \rightarrow 1$.

II Part II: Horizons and Discounting

Q5: A key structural choice in any MDP is whether the agent acts for a fixed number of steps or indefinitely. ⁶

(a) What is a *finite horizon* MDP, and why may the optimal policy be *non-stationary*? What is an *infinite horizon* MDP, and why is the optimal policy *stationary* in that setting?

⁶08-MDP-Bellman, Page 5

(b) Infinite-horizon MDPs require a discount factor $\gamma \in [0, 1)$. Using the Robot Vacuum MDP from Q4, demonstrate concretely why $\gamma = 1$ causes a problem for two scenarios:

(i) Write the undiscounted total reward for an agent stuck in Broken forever, and for an agent cycling optimally between Clean and Messy forever. Why can these not be compared?

(ii) Now consider an agent that repeatedly uses Force from Jammed. 50% of the time it escapes to Clean (positive total reward), and 50% of the time it reaches Broken (negative total reward). Why does this make the expected undiscounted return of Force undefined when $\gamma = 1$?

(c) Discounting is justified two ways. First, as a *preference for sooner rewards*. Second, as a model of *uncertain termination*. Describe both interpretations in your own words.

(d) Write a closed-form expression for $V^*(\text{Broken})$ in terms of γ . Show the derivation using the Bellman equation for the absorbing state. What happens to this value as $\gamma \rightarrow 1$, and why does this matter for Jammed's Force action?

(e) The discount parameter γ directly controls the agent's patience. For the Jammed state specifically:

- When $\gamma \rightarrow 0$: Write the approximate values of $Q(\text{Jammed}, \text{Reset})$ and $Q(\text{Jammed}, \text{Force})$ (the future terms vanish). Which action does the myopic agent select and why?
- When $\gamma \rightarrow 1$: Use your closed-form $V^*(\text{Broken})$ from (d) to write $Q^*(\text{Jammed}, \text{Force})$ explicitly. What happens to this Q-value as $\gamma \rightarrow 1$? Compare to $Q^*(\text{Jammed}, \text{Reset})$ and state which action now dominates.

III Part III: The Bellman Equation

Q6: Define the *optimal value function* $V^*(s)$ and the *Q-value* (action-value) $Q^*(s, a)$.⁷

(a) Write the formula for $Q^*(s, a)$ in full, identifying what each term represents.

⁷Course notes: 08-MDP-Bellman, Page 8

(b) Express $V^*(s)$ in terms of $Q^*(s, a)$. Then write the optimal policy $\pi^*(s)$ in terms of $Q^*(s, a)$.

(c) Combine your answers to derive the *Bellman equation* for $V^*(s)$. Label which part represents the agent's choice, which part represents the environment's randomness, and which part captures the recursive structure.

Q7: Using the Robot Vacuum MDP from Q4 with $\gamma = 0.9$, perform two rounds of value iteration starting from $V^0(s) = 0$ for all states. The update rule is $V^{k+1}(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^k(s')]$.⁸

(a) Compute $V^1(s)$ for all four states. Note that Broken has only one action (Wait), so no max is needed there. For Clean and Messy, show both Q-values. For Jammed, compare Reset and Force explicitly.

⁸Course notes: 08-MDP-Bellman, Pages 9–10

(b) Compute $V^2(s)$ for all four states. Again show all Q-values at each non-absorbing state. *Hint: notice how $V^1(\text{Broken}) = -20$ now feeds into $Q^2(\text{Jammed}, \text{Force})$ via the $0.5 \cdot V^1(\text{Broken})$ term.*

(c) At Jammed, which action did the max select in round 1? Which does it select in round 2? The answer changes, explain intuitively why in terms of what $V^1(\text{Broken})$ contributes to Force's Q-value once it is no longer zero.

Q8: Now solve for the *true* $V^*(s)$ directly as a linear system. The optimal policy from Q7 (once estimates settled) is $\pi^* = \{\text{Clean} : \text{Patrol}, \text{Messy} : \text{Explore}, \text{Jammed} : \text{Reset}\}$. Because the policy is known, substitute the chosen action at each state and drop the max (this turns the recursive Bellman equation into a solvable linear system).⁹

(a) Solve for $V^*(\text{Broken})$. Broken is terminal with only Wait available, write $V^*(\text{Broken}) = R + \gamma \cdot V^*(\text{Broken})$, then solve for $V^*(\text{Broken})$ in terms of γ . Plug in $\gamma = 0.9$.

⁹Course notes: 08-MDP-Bellman, Page 10

(b) Write the Bellman equation for $V^*(\text{Jammed})$ under Reset. Reset transitions deterministically to Messy, so Broken does *not* appear here. You will get: $V^*(\text{Jammed}) = -5 + 0.9 \cdot V^*(\text{Messy})$.¹⁰

¹⁰*Don't solve yet*, keep this as an expression; you'll substitute it in part (c)

(c) Write the Bellman equation for $V^*(\text{Clean})$ under Patrol. Collect $V^*(\text{Clean})$ on the left to get the form: $\alpha \cdot V^*(\text{Clean}) = c_1 + \beta \cdot V^*(\text{Messy})$. (Identify α , c_1 , β .)

(d) Write the Bellman equation for $V^*(\text{Messy})$ under Explore. The Explore transitions go to Clean (40%), Messy (30%), and Jammed (30%). Substitute your expression from (b) to eliminate $V^*(\text{Jammed})$, then collect $V^*(\text{Messy})$ on the left. You should arrive at: $\delta \cdot V^*(\text{Messy}) = c_2 + \varepsilon \cdot V^*(\text{Clean})$. (Identify δ , c_2 , ε .)

(e) You now have two equations in $V^*(\text{Clean})$ and $V^*(\text{Messy})$ from (c) and (d). Solve this 2×2 linear system (substitute one into the other). Then use (b) to find $V^*(\text{Jammed})$.

(f) Verify: compute $Q^*(\text{Jammed}, \text{Force})$ and $Q^*(\text{Messy}, \text{Vacuum})$ using your exact values. Confirm Reset beats Force at Jammed and Explore beats Vacuum at Messy.

Q9: The structure of the Bellman equation should look familiar from earlier in the course. ¹¹

(a) Identify the correspondence between the Bellman equation and an expectimax tree. What do the agent (square) nodes correspond to? What do the Q-state (circle) nodes correspond to?

¹¹Course notes: 08-MDP-Bellman,
Page 11

(b) Expectimax expands a tree from a single root, which cannot handle cycles without unrolling into an infinite recursion. How does the Bellman equation handle the same problem? What makes it more efficient?

IV Part IV: Value and Policy Iteration

Q10: Value iteration finds the optimal value function by repeatedly applying the Bellman equation as an update rule, without ever fixing a policy in advance.¹²

(a) Write the value iteration update rule $V^{k+1}(s)$ in full. Why does keeping the max inside the update rule allow the algorithm to work without knowing the optimal policy ahead of time?

¹²Course notes: 08-MDP-Bellman,
Pages 9–10

(b) Value iteration is guaranteed to converge for any finite state space when $\gamma < 1$. Explain the contraction argument: why does each round shrink the error between V^k and V^* by at least a factor of γ ? What does this say about convergence speed as $\gamma \rightarrow 1$?

(c) After convergence, how do you extract the optimal policy π^* from the converged values V^* ? Write the formula. Why does this work, even though π^* was never explicitly tracked during the iterations?

(d) Consider the relationship between value convergence and policy convergence. In the Roomba MDP from Q4, the action selected at Jammed flipped between rounds 1 and 2 of your value iteration (Q7c). Does the policy need to be fully stable for the values to be correct? Explain what “the relative ordering of actions is often correct early on” means in practice.

Q11: Policy iteration is an alternative algorithm that alternates between two steps: *policy evaluation* and *policy improvement*.¹³ Describe both steps precisely:

- *Policy evaluation*: Given a fixed policy π , what system of equations do you solve? Why is there no max in this system, and what does that allow you to do?

¹³Course notes: 08-MDP-Bellman, Pages 12–13

- *Policy improvement*: Given the values V^π from evaluation, how do you compute a new policy π' ? Write the formula. What does it mean if $\pi' = \pi$?

Q12: Using the Robot Vacuum MDP from Q4 with $\gamma = 0.9$, run one full iteration of policy iteration starting from the all-safe policy $\pi^0 = \{\text{Clean : Charge, Messy : Vacuum, Jammed : Reset}\}$.

(a) *Policy evaluation:* Broken is absorbing with $V^*(\text{Broken}) = -200$. For the remaining states, write and solve the three linear equations under π^0 . Show how you eliminate variables.

(b) *Policy improvement:* Using your V^{π^0} values, compute all Q-values for every non-absorbing state. Fill in the table:

State	Action	$Q(s, a)$ under V^{π^0}
Clean	Patrol	
Clean	Charge	
Messy	Vacuum	
Messy	Explore	
Jammed	Reset	
Jammed	Force	

What is the improved policy π^1 ? Has it changed from π^0 ?

(c) If $\pi^1 \neq \pi^0$, one more round of policy evaluation is required. Assuming the policy has stabilized after this round, how would you verify that π^1 is optimal without running another improvement step?

Q13: Compare and contrast value iteration and policy iteration.¹⁴

(a) Both algorithms are guaranteed to converge to the same V^* and π^* . Describe the key structural difference in *how* each algorithm reaches that result. In one sentence each: what does value iteration commit to at each step, and what does policy iteration commit to?

¹⁴Course notes: 08-MDP-Bellman,
Page 13

(b) Policy iteration is often said to converge in fewer *iterations* than value iteration, even though each iteration of policy iteration is more expensive. Explain why:

- Policy iteration converges in a finite number of iterations (hint: what is the bound, and why must it terminate?).
- Value iteration converges asymptotically but may take many rounds before V^k is close enough to V^* to extract the correct policy.

(c) Value iteration requires knowing T and R upfront. What algorithmic family handles the case where the agent must *learn* these from experience instead of having them given?

V Part V: Reinforcement Learning

Q14: Reinforcement learning addresses the setting where the agent still has an MDP but does not know T or R .¹⁵ List the four components of the MDP that still exist in the RL setting. What is the one critical difference from the planning setting?

¹⁵Course notes: 09-Reinforcement-Learning, Page 1

Q15: Distinguish *offline* learning from *online* learning. In the RL lecture, what analogy was used to illustrate the difference? Which mode do value iteration and policy iteration belong to, and why?

Q16: What are the three central challenges of online RL. Explain them in your own words.

Q16: In model-based RL the agent estimates the MDP from observed experience, then solves it with standard planning.¹⁶

(a) Define what a *sample* and an *episode* are in this context. Write the notation for each from the lecture.

¹⁶Course notes: 09-Reinforcement-Learning, Pages 2–3

(b) Describe the three-step model-based RL procedure. Be precise about how the transition counts are converted into a valid distribution and when the discount factor γ enters.

(c) Musty the mustang claims: “When estimating $\hat{T}(s, a, s')$ from experience, you must keep samples grouped by episode — mixing transitions across episodes gives you a corrupted transition function.” Is Musty correct? Cite the concept that justifies your answer. Then identify one situation where Musty *would* be right and episode boundaries do matter.

Q17: Consider the following four episodes in an MDP with states A, B, C, D, E and terminal T :

Episode	Trace
1	$B \rightarrow C, -1 \quad C \rightarrow D, -1 \quad D \rightarrow T, +10$
2	$B \rightarrow C, -1 \quad C \rightarrow D, -1 \quad D \rightarrow T, +10$
3	$E \rightarrow C, -1 \quad C \rightarrow D, -1 \quad D \rightarrow T, +10$
4	$E \rightarrow C, -1 \quad C \rightarrow A, -1 \quad A \rightarrow T, -10$

(a) From these samples, estimate $\hat{T}(C, \text{down}, D)$ and $\hat{T}(C, \text{down}, A)$. Show your count-based calculation.

(b) Estimate $\hat{R}(D, \text{exit}, T)$ and $\hat{R}(A, \text{exit}, T)$.

(c) Given these estimates and assuming $\gamma = 0.9$, what is the model-based agent's optimal policy? (You do not need to solve the full linear system, reason qualitatively from the reward structure.)

Q18: Model-free RL avoids storing a full transition model by learning values directly from samples.¹⁷ With that in mind, what are the three ways to compute an expected value $E[X]$. Write all three formulas and identify which corresponds to the model-based, model-free, and exact approaches.

¹⁷Course notes: 09-Reinforcement-Learning, Pages 4–5

Q19: Describe *Direct Evaluation*. What policy does the agent follow, what does it record, and how does it produce $\hat{V}^\pi$? State the key weakness of direct evaluation that motivates TD learning.

Q20: *Temporal-Difference Learning* (TD learning) fixes the weakness you identified in (Q19). Write:

- (a) The formula for the *sample value* $V_{\text{sample}(s)}^\pi$ implied by a single transition (s, a, s', r) .

(b) The exponential moving-average update rule for $\hat{V}^{\pi(s)}$. Identify the *learning rate* α and explain what it controls. What does the update reduce to when $\alpha = 0$? When $\alpha = 1$?

(c) Why does TD learning converge faster than direct evaluation? What structural fact about MDPs does it exploit that direct evaluation ignores?

Q21: Q-learning extends TD learning to learn optimal policies without a fixed policy.¹⁸

(a) Explain why TD learning cannot be directly used to find an *optimal* policy. What would you have to do each time the policy changes under pure TD learning?

¹⁸Course notes: 09-Reinforcement-Learning, Pages 5–6

(b) Write the Q-learning update rule for a transition (s, a, s', r) . Label every term.

(c) Compare the Q-learning update to the TD learning update. What single structural change makes Q-learning *off-policy* while TD learning is *on-policy*? Where does the max appear, and why does that matter?

(d) Once Q-values have converged, how do you extract the policy $\pi(s)$? Write the formula.

(e) Q-learning is guaranteed to converge to the optimal policy given two conditions. State both conditions. Why does “it does not matter how you select actions” hold in the limit, even if early actions are suboptimal?

(f) An agent doing Q-learning in an unknown environment with states A, B, C , terminal T , and actions $\leftarrow$ and $\rightarrow$ observes the following repeating sequence (with $\alpha = 0.5$, $\gamma = 0.9$, all Q-values initialized to 0):

s	a	s'	r
A	$\rightarrow$	B	0
B	$\rightarrow$	C	0
C	$\leftarrow$	T	1

After which sample does $Q(C, \leftarrow)$ first become nonzero? After which sample does $Q(B, \rightarrow)$ first become nonzero? Trace the update for each.